Agentic Computational Reproducibility for Published Research

CSE598 (Agentic AI) — Capstone Project Proposal

Student name	Jason Lunder
Project title	Agentic Computational Reproducibility of Published Research on CORE-Bench
Repository / notebook link	**[TODO: GitHub repository URL]**
Configuration location	`examples/` and `.env.example`

1 Problem Definition

Published computational research is often not reproducible: code capsules accompanying a paper fail to run months or years later because of dependency rot, undocumented environments, or ambiguous run instructions. Manually verifying a paper's numerical claims requires a human to install the right toolchain, execute the code, and cross-check outputs against the text — a slow process that does not scale to the volume of published work. This project builds an *agentic system* that automates that verification: given a paper's code capsule (source code plus a README and/or Dockerfile), the agent must autonomously install dependencies, execute the code, and extract the paper's reported numerical results in machine-checkable form.

2 Motivation and Project Scope

We target CORE-Bench [Siegel et al., 2024b,a]: 270 tasks from 90 published papers in computer science, social science, and medicine (Python and R), split 45 train / 45 test papers, each evaluated at three difficulty tiers (135 tasks per split). Per the benchmark's harness prompts: *Easy* gives the agent the complete code output from a successful run and forbids execution, requiring only information extraction; *Medium* gives the Dockerfile plus a README and requires running the provided Docker command; *Hard* gives only the README with no Dockerfile, requiring the agent to install dependencies and determine the run commands itself. On the 135-task *test* split, the best published agent (CORE-Agent, GPT-4o) reaches only 21.48% task accuracy on the hard tier [Siegel et al., 2024b] — "showing the vast scope for improvement in automating routine scientific tasks." The capstone goal is an iterative pipeline that improves on that number; this proposal covers the *baseline* against which it will be measured. **Scope IN:** Python capsules, CPU-only, small capsules (roughly two-thirds of the test split is under 26 MB). **Scope OUT (deferred):** R capsules, GPU tasks, vision/figure questions, multi-gigabyte capsules, Azure parallel execution.

3 Runnable Baseline

The baseline is a deliberately *strict, single-pass, no-retry* pipeline: (1) fetch the capsule, (2) read the README, (3) issue *one* LLM call proposing install and run commands, (4) execute those commands *once* in Docker, (5) issue *one* LLM call to extract answers from the captured stdout, (6) write `report.json`, and (7) score it with the vendored official scorer. No retries, self-correction, or re-planning are permitted at any stage.

This design is intentional and is the intellectual core of the project: the baseline is a *scientific control*. The capstone system will add an iterative loop — observing failures and retrying, revising commands, or re-reading documentation. Without a strict single-pass floor, any accuracy gain from that loop would be confounded with gains from simply using a better model or more prompt engineering. Holding the model, prompts, and one-shot budget fixed and varying only "iterate or not" isolates the marginal value the loop contributes. The single-pass design also attributes every failure to exactly one pipeline stage (install, execution, or extraction) — a useful diagnostic for where the adaptive system needs to invest effort. LLM access is provider-agnostic via `litellm`, so the pipeline runs unmodified with either an Anthropic or an OpenAI API key.

4 Test Case and Baseline Output

As a running example we use capsule `capsule-4180912` (easy tier), where the agent is given a populated `results/` directory and must answer the task's questions by reading it alone, with code execution explicitly forbidden. The baseline writes a `report.json` per task containing the extracted answers, the raw stdout/stderr captured from the container (medium/hard tiers), and pipeline timing; the vendored scorer then compares each answer to its ground-truth 95% prediction interval (or does an exact/case-insensitive string or list match) and marks the task correct only if *every* question in it is correct.

[TODO: insert baseline_output screenshot and observed answers for capsule-4180912]

5 Reproducibility and Run Instructions

The repository pins dependencies in `requirements.txt` and reads provider credentials from `.env.example` (copy to `.env` and set exactly one of `ANTHROPIC_API_KEY` or `OPENAI_API_KEY`). Capsule metadata and prompt templates live under `examples/`. A single task is run with:

```
python run_baseline.py --capsule capsule-4180912 --tier easy
```

which fetches the capsule, runs the single-pass pipeline described in Section 3, and writes results under `results/`. Batches over a subset/tier/split are supported via `-subset`, `-tier`, and `-limit` flags (see `run_baseline.py -help`).

6 Initial Evaluation Plan

We will compare the future iterative agentic system against this single-pass baseline on: (i) official task accuracy (all-or-nothing per task), (ii) per-question accuracy, (iii) wall-clock latency, (iv) dollar cost per task, and (v) a failure-stage breakdown (install vs. execution vs. extraction) to show *where* iteration recovers failures the baseline cannot. Cost is informative: successful CORE-Agent (GPT-4o) tasks cost $0.54 on average versus $2.59 for failed tasks — failure is more expensive than success, motivating early failure detection. The published numbers below (test split, 45 tasks/tier) are our external reference point; the benchmark reports no human/expert baseline.

Agent	Model	Easy	Medium	Hard
CORE-Agent	GPT-4o	60.00%	57.78%	21.48%
CORE-Agent	GPT-4o-mini	44.44%	32.59%	16.30%
AutoGPT	GPT-4o	35.56%	37.78%	6.67%
AutoGPT	GPT-4o-mini	8.89%	2.22%	2.22%

Source: Siegel et al. [2024b], Table 5 (pass@1, test split, 45 papers/tier).

Our own single-pass baseline numbers on the same metrics are pending measurement: **[TODO: easy-tier task accuracy]**, **[TODO: medium-tier task accuracy]**, **[TODO: hard-tier task accuracy]**, **[TODO: per-question accuracy]**, **[TODO: mean wall-clock latency]**, **[TODO: mean $ cost per task]**, **[TODO: failure-stage breakdown]**.

7 Limitations and Next Steps

Several risks are known up front. *Dependency rot*: years-old research code often depends on unpinned or deprecated packages, so installation may fail for reasons unrelated to agent competence. *Resource scope*: capsules needing GPUs or multi-gigabyte data are out of scope for this phase, undercounting the true task pool. *Language coverage*: R capsules (roughly half of CORE-Bench) are deferred, so results here characterize only the Python subset. *Stochastic scoring risk*: grading uses a 95% prediction interval from three ground-truth runs, so even a correct, faithfully reproduced run can fall outside it by chance — a perfect agent would not score 100%. Next steps: run the baseline across the full small/Python/CPU test subset, use the Section 6 failure-stage breakdown to prioritize what the iterative loop targets first, then extend to R capsules and larger resource budgets.

References

Zachary S. Siegel, Sayash Kapoor, Nitya Nadgir, Benedikt Stroebl, and Arvind Narayanan. CORE-Bench GitHub repository. `https://github.com/siegelz/core-bench`, 2024a. Accessed 2026-09-06.

Zachary S. Siegel, Sayash Kapoor, Nitya Nadgir, Benedikt Stroebl, and Arvind Narayanan. CORE-Bench: Fostering the credibility of published research through a computational reproducibility agent benchmark. *arXiv preprint arXiv:2409.11363*, 2024b. URL `https://arxiv.org/abs/2409.11363`.